
Image processing project

Microprocessor Application– Team project

7조

IT융합전공 김동준

IT융합전공 안덕찬

IT융합전공 이장근

전자공학전공 우상욱



목차

01 프로젝트 개요

02 블록 다이어그램 및 실행결과

03 메모리 재정렬

04 ARM code level 최적화

05 최적화 전후 성능 비교

06 Trouble shooting



01

Red pixel count

02

RGB convert

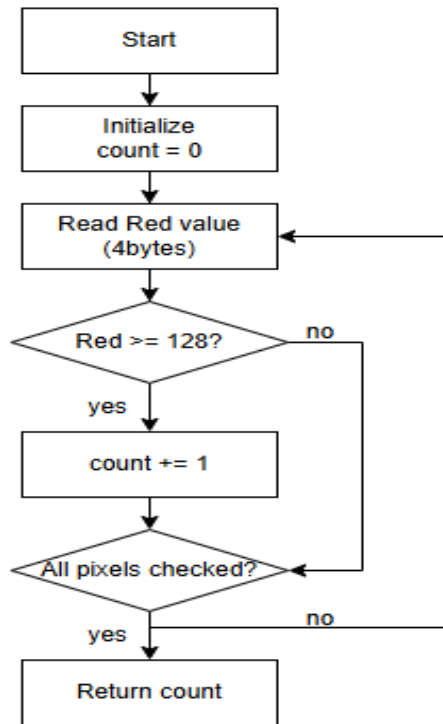
03

Convert GrayScale

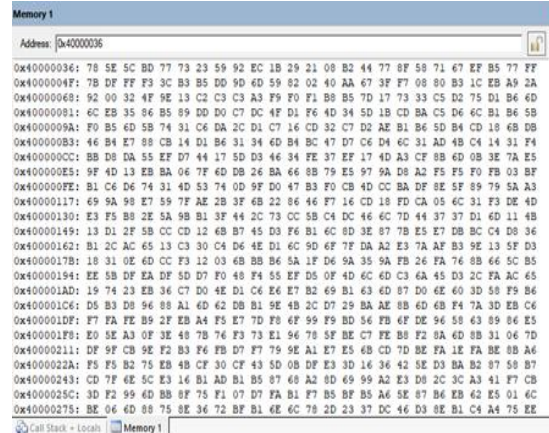
04

Memory relocation & ARM optimization

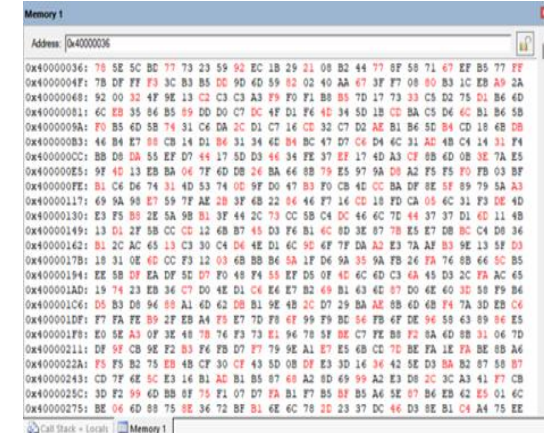
2-1. Red Pixel Count



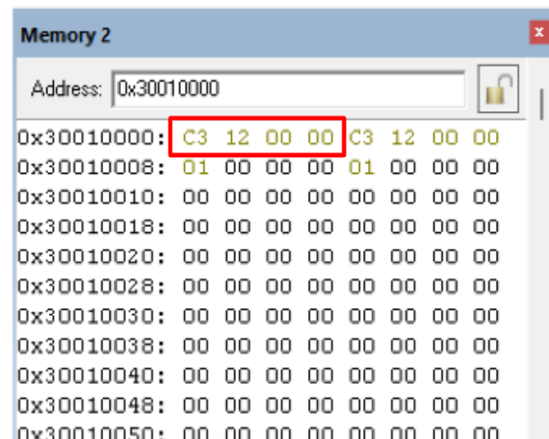
[Red Pixel Count 블록다이어그램]



[실행 전 메모리맵]



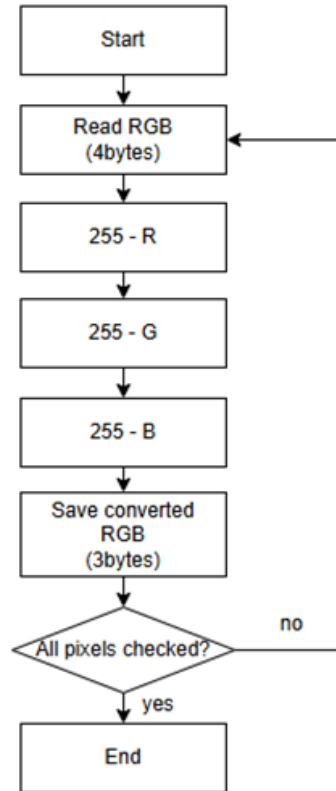
[실행 후 메모리맵]



[0X30010000에 저장된 값]



2-2. Convert RGB



[Convert RGB 블록다이어그램]

Memory 3

Address: 0x30040000

0x30040000:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x30040012:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x30040024:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x30040036:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x30040048:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x3004005A:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x3004006C:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x3004007E:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x30040090:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400A2:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400B4:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400C6:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400D8:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400EA:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x300400FC:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
0x3004010E:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

[실행 전 메모리맵]

Memory 3

Address: 0x30040000

0x30040000:	87	A1	A3	88	8C	DC	6D	13	E4	DE	F7	4D	88	70	A7	98	10	4A
0x30040012:	00	84	20	0C	C3	4C	22	62	92	7D	FD	BF	98	C0	08	7F	4C	E3
0x30040024:	56	D5	6D	CD	B0	61	3D	3C	06	0F	0E	4A	82	E8	CC	3A	2D	
0x30040036:	2E	49	92	14	CA	79	76	22	2F	23	B0	2E	B2	CB	A2	32	45	3A
0x30040048:	93	4E	49	0F	4A	92	8B	CE	39	D3	2E	38	32	CD	38	51	4E	49
0x3004005A:	4B	32	E7	24	B9	4B	77	34	EB	49	CE	CB	4B	43	B8	39	2B	93
0x3004006C:	52	B4	3B	CE	0B	44	25	AA	10	BB	E8	A2	B9	C8	01	10	E8	B2
0x3004007E:	30	74	92	C1	85	1A	B2	EC	14	F9	80	92	D9	45	99	86	1A	68
0x30040090:	27	5D	0A	0F	04	FC	4E	39	29	CE	B2	AC	F2	60	2F	4C	0F	34
0x300400A2:	33	45	20	A0	76	86	5C	96	65	18	A6	80	D4	C0	94	79	B9	08
0x300400B4:	32	E7	02	FA	93	CE	21	B2	1C	47	D1	A5	4E	C0	BB	8C	33	A4
0x300400C6:	23	B9	93	BB	C8	C8	92	EE	B4	2E	D0	A4	32	ED	94	BA	2C	09
0x300400D8:	93	72	C1	84	1A	18	43	3B	27	4E	D3	53	EC	3C	CF	29	B1	2E
0x300400EA:	62	90	80	5D	1C	85	4C	61	EC	2C	E7	CE	92	33	0C	FC	94	44
0x300400FC:	A5	E0	29	CA	65	04	05	89	74	A3	4A	11	20	15	20	28	0F	B7
0x3004010E:	AA	10	2A	B2	93	92	95	BA	2C	05	53	9A	8B	DC	14	38	2F	B1

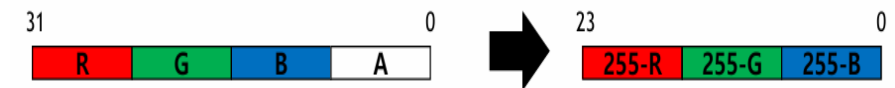
[실행 후 메모리맵]

Memory 2

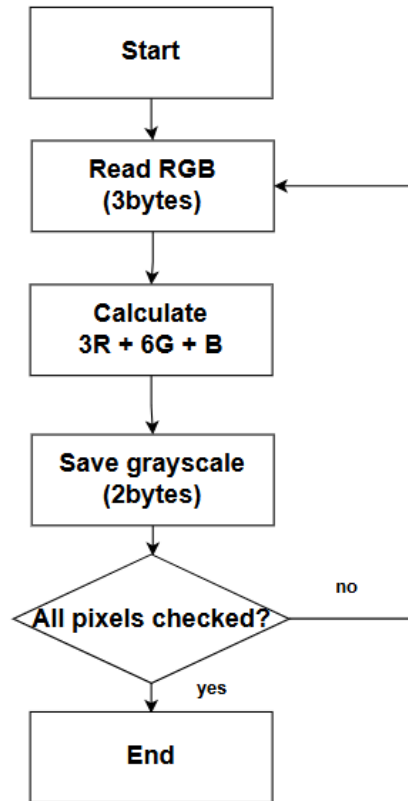
Address: 0x30010000

0x30010000:	C3	12	00	00	C3	12	00	00
0x30010008:	01	00	00	00	01	00	00	00
0x30010010:	00	00	00	00	00	00	00	00
0x30010018:	00	00	00	00	00	00	00	00
0x30010020:	00	00	00	00	00	00	00	00
0x30010028:	00	00	00	00	00	00	00	00
0x30010030:	00	00	00	00	00	00	00	00
0x30010038:	00	00	00	00	00	00	00	00
0x30010040:	00	00	00	00	00	00	00	00
0x30010048:	00	00	00	00	00	00	00	00
0x30010050:	00	00	00	00	00	00	00	00

[bool isSame 실행결과]



2-3. Grayscale



[Grayscale 블록다이어그램]

Memory 4

Address: 0x30050000

0x30050000:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050012:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050024:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050036:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050048:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3005005A:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3005006C:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3005007E:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050090:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500A2:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500B4:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500C6:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500D8:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500EA:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x300500FC:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x3005010E:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x30050120:	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

[실행 전 메모리맵]

Memory 2

Address: 0x30020000

0x30020000:	F8 03 3A 04 59 07 45 01 17 05 84 07 BE 06 F4 04 B2 06
0x30020012:	D2 01 A6 03 CE 05 89 03 0E 03 9B 07 7C 09 24 05 09 06
0x30020024:	24 07 85 04 99 07 3F 05 7C 02 88 07 20 06 7B 07 48 03
0x30020036:	31 06 5A 04 E6 06 02 07 E9 04 6E 06 7C 03 CB 06 B6 07
0x30020048:	8D 04 06 07 7B 05 B3 01 08 03 A4 03 1C 06 7B 04 44 02
0x3002005A:	79 03 34 05 60 07 49 07 B5 08 8D 07 B4 02 B1 04 84 08
0x3002006C:	9F 07 CC 04 F9 04 4A 05 66 02 2D 04 F4 03 C8 02 4B 05
0x3002007E:	96 03 D1 03 7C 06 A4 04 4D 02 F8 01 E8 03 3E 03 B7 06
0x30020090:	D0 04 B6 07 A4 07 C7 03 FB 04 27 05 F0 04 B2 07 E0 05
0x300200A2:	3A 03 02 07 46 03 9E 02 36 05 3D 06 40 06 F8 08 6D 08
0x300200B4:	6E 07 DC 03 AF 03 5B 07 19 03 83 07 9D 08 C4 05 E3 06
0x300200C6:	C0 03 89 07 CB 05 55 07 B7 05 B9 05 F5 05 16 09 30 04
0x300200D8:	C6 08 DC 05 53 07 35 04 D0 07 81 02 28 05 9F 06 E2 07
0x300200EA:	7F 06 34 01 3D 07 D8 08 59 06 7E 08 E5 06 ED 07 4F 09
0x300200FC:	52 05 5C 04 2C 08 8C 02 E1 06 72 07 7C 06 E2 06 91 05
0x3002010E:	7A 06 8F 02 E4 07 14 05 0C 07 0B 08 21 07 C4 06 21 03
0x30020120:	38 03 60 03 19 06 00 01 F3 06 60 06 80 04 C7 01 E6 03

[실행 후 메모리맵]

Memory 2

Address: 0x30010000

0x30010000:	C3 12 00 00 C3 12 00 00
0x30010008:	01 00 00 00 01 00 00 00
0x30010010:	00 00 00 00 00 00 00 00
0x30010018:	00 00 00 00 00 00 00 00
0x30010020:	00 00 00 00 00 00 00 00
0x30010028:	00 00 00 00 00 00 00 00
0x30010030:	00 00 00 00 00 00 00 00
0x30010038:	00 00 00 00 00 00 00 00
0x30010040:	00 00 00 00 00 00 00 00
0x30010048:	00 00 00 00 00 00 00 00
0x30010050:	00 00 00 00 00 00 00 00

[bool isSame 실행결과]



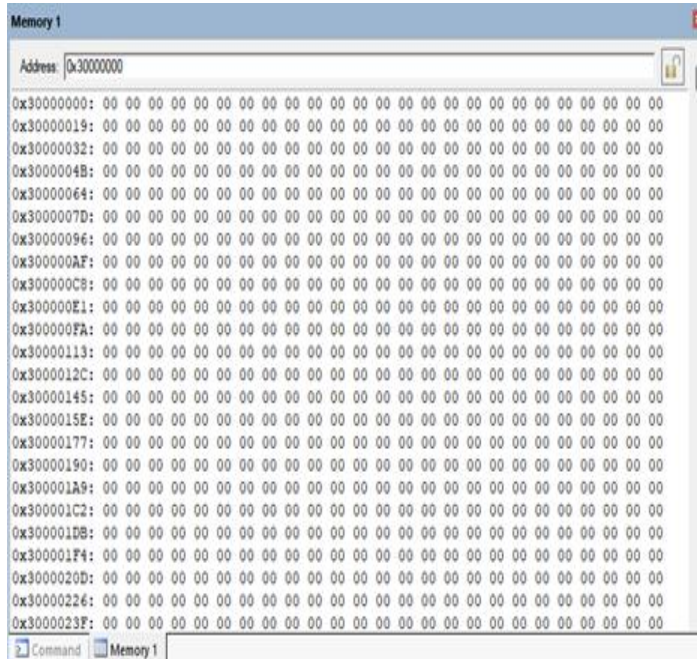
메모리 재정렬 방식

R R R R R ...

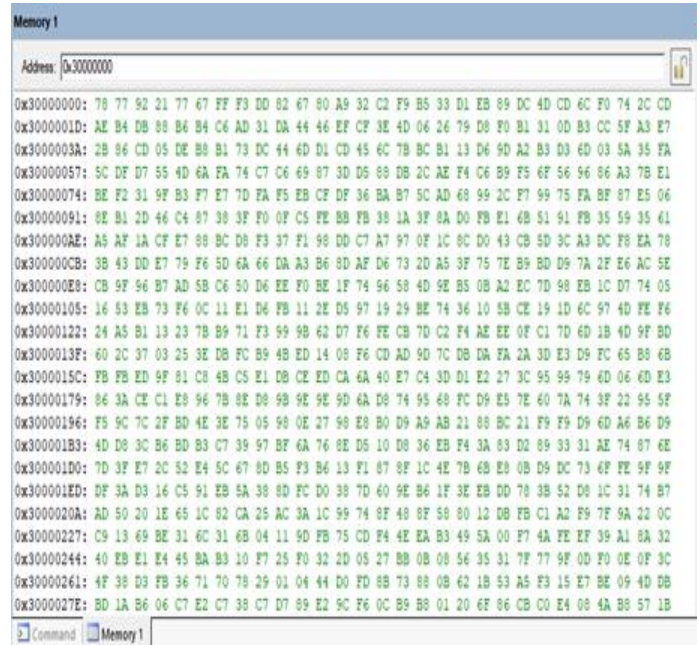
G G G G G ...

B B B B B ...

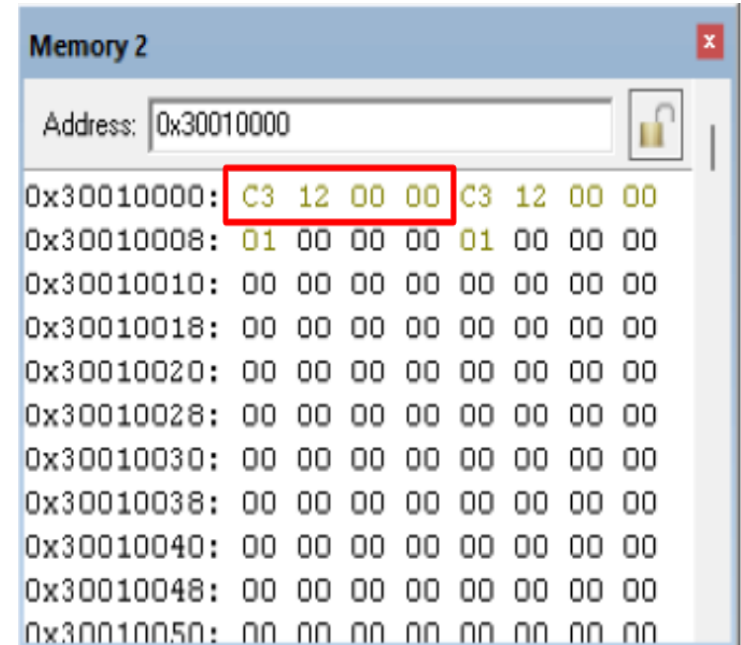
3-1. Red Pixel Count (relocation)



[실행 전 메모리맵]

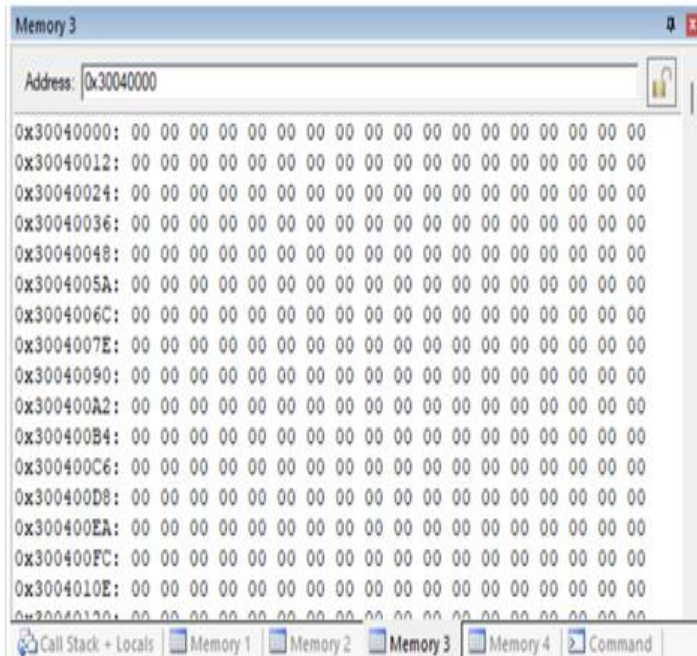


[실행 후 메모리맵]

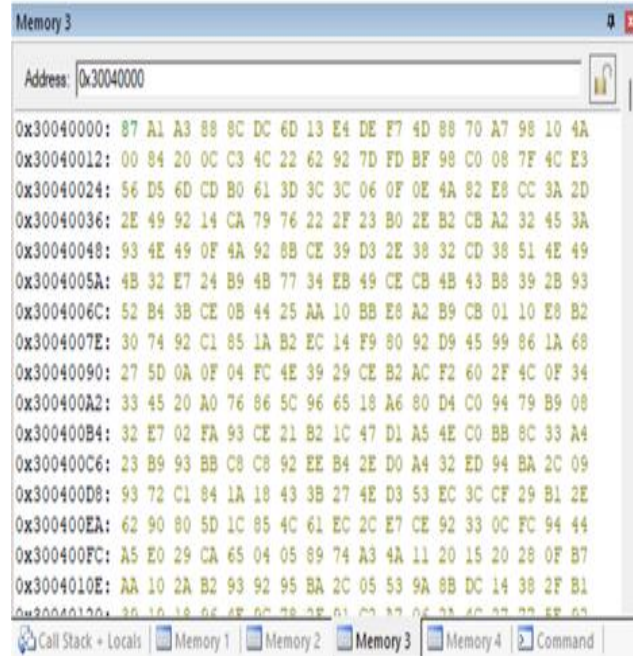


[0X30010000에 저장된 값]

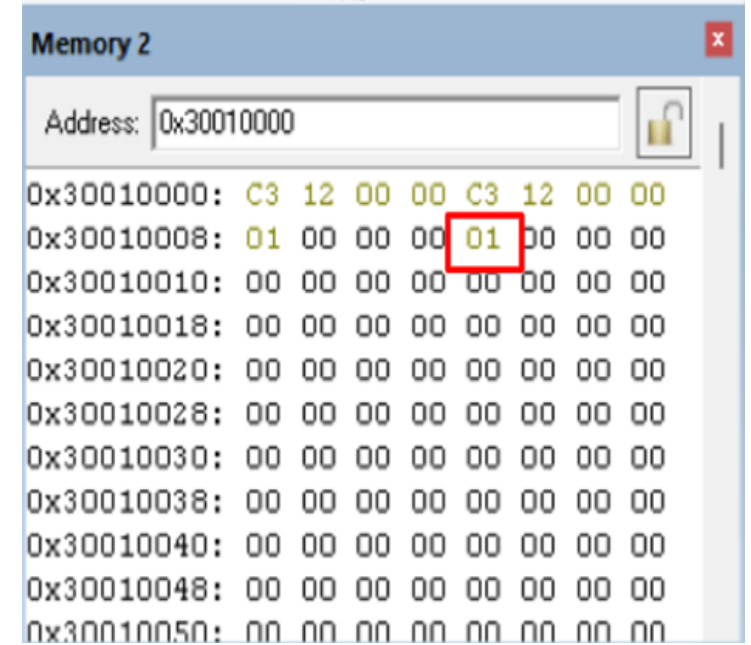
3-2. RGB convert (relocation)



[실행 전 메모리맵]

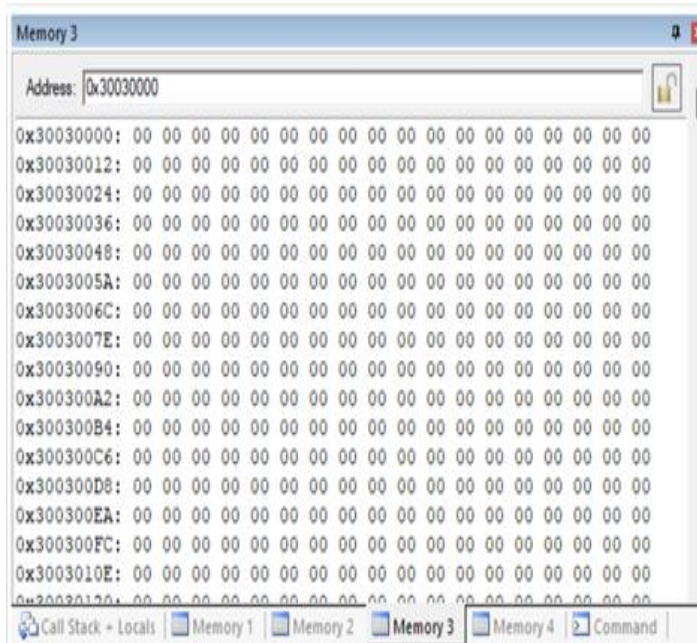


[실행 후 메모리맵]

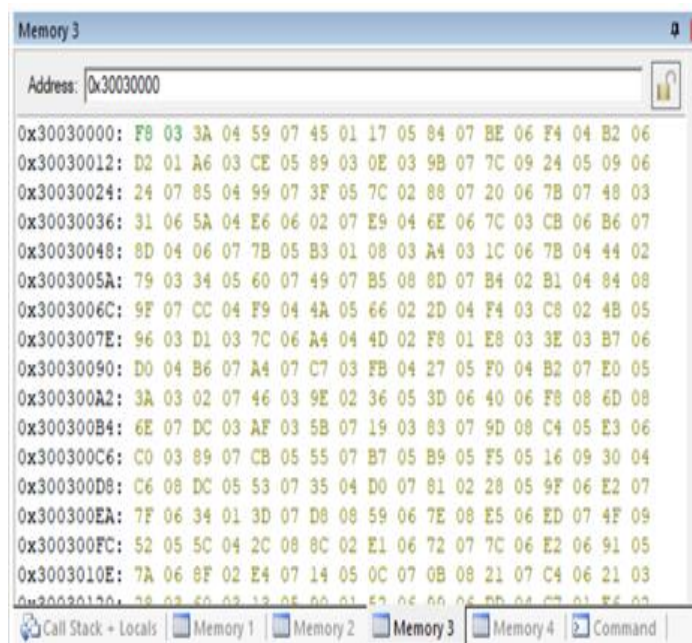


[bool isSame 실행결과]

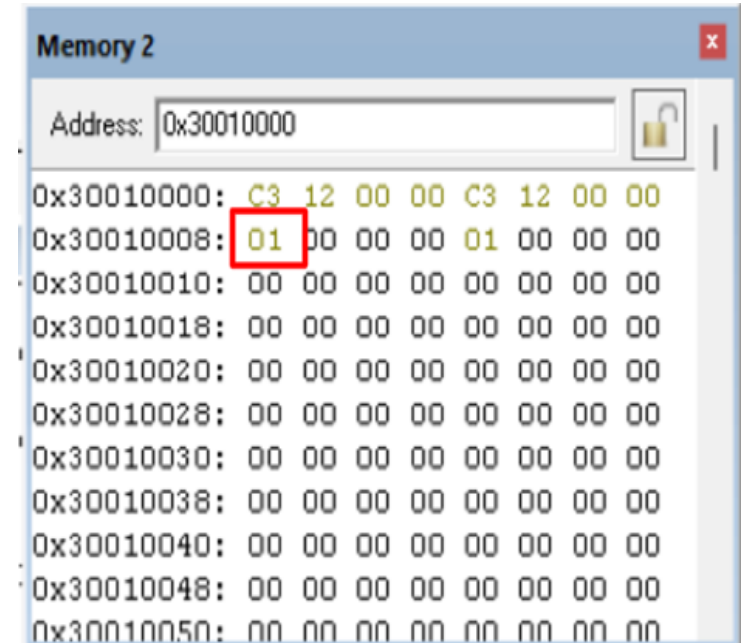
3-3. Grayscale (relocation)



[실행 전 메모리맵]



[실행 후 메모리맵]



[bool isSame 실행결과]

3-4. 재정렬 전후 성능 비교

Performance Analyzer

Reset Show: Modules

Module/Function	Calls	Time(S...	Time(%)
ma_project_dj		33,371 ms	100%
source files/convertGray.c		12,193 ms	37%
countRed	1	2,976 ms	9%
convertReverse	1	4,608 ms	14%
convertGray	1	4,608 ms	14%
source files/main.c		43,020 us	0%
source files/find_IDAT.c		11,580 us	0%
Startup.s		1,660 us	0%
.../Desktop/files/serial.c		0,380 us	0%
source files/isSame.c		11,521 ms	35%
source files/countRed.s		1,728 ms	5%
__asm_0x4C0	1	1,728 ms	5%
source files/convertGray.s		3,648 ms	11%
__asm_0x4E0	1	3,648 ms	11%
source files/convertReverse.s		4,224 ms	13%
__asm_0x510	1	4,224 ms	13%
source files/convertReverse...		0us	0%
source files/countRedRel.s		0us	0%
source files/convertGrayRel.s		0us	0%

[기존 Performance Analyzer 캡처]

Performance Analyzer

Reset Show: Modules

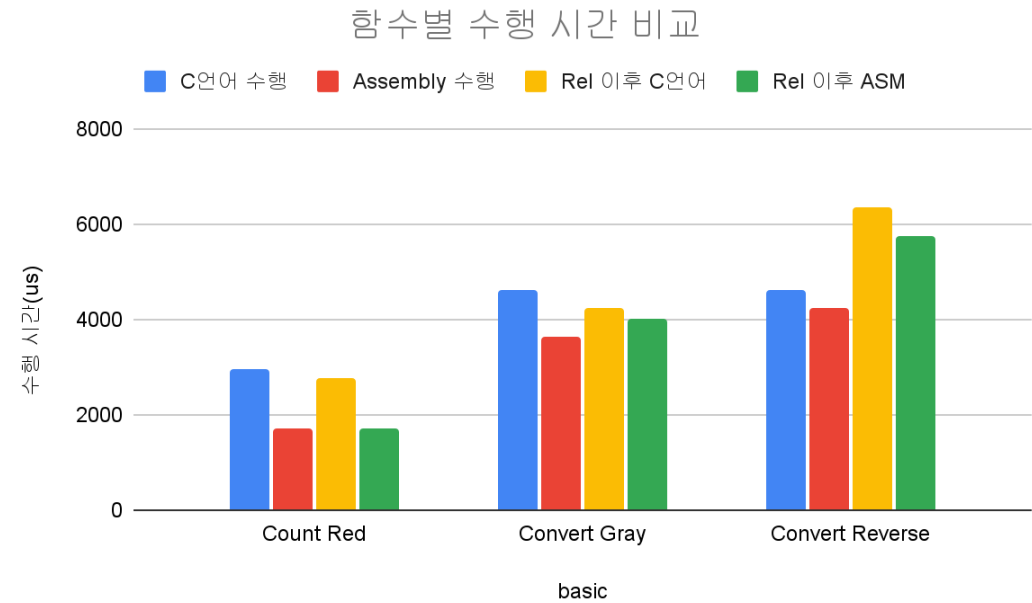
Module/Function	Calls	Time(S...	Time(%)
ma_project_dj		40,477 ms	100%
source files/find_IDAT.c		53,020 us	0%
Startup.s		1,660 us	0%
.../Desktop/files/serial.c		0,380 us	0%
first grayrel.s		4,032 ms	10%
__asm_0x8C4	1	4,032 ms	10%
first reverserel.s		5,760 ms	14%
__asm_0x8A8	1	5,760 ms	14%
firstcountrel.s		1,728 ms	4%
__asm_0x888	1	1,728 ms	4%
source files/isSame.c		11,521 ms	28%
source files/main_reallo.c		2,740 us	0%
source files/convertReverse.s		0us	0%
source files/convertGray.s		0us	0%
source files/countRed.s		0us	0%
source files/reAllocation.c		17,378 ms	43%
countRedRel	1	2,784 ms	7%
convertReverseRel	1	6,336 ms	16%
convertGrayRel	1	4,225 ms	10%
reAllocation	1	4,032 ms	10%
source files/convertReverse...		0us	0%
source files/countRedRel.s		0us	0%
source files/convertGrayRel.s		0us	0%

[재정렬 Performance Analyzer 캡처]



3-4. 재정렬 전후 성능 비교

(us)	C언어 수행	Assembly 수행	Rel 이후 C언어	Rel 이후 ASM
Count Red	2976	1728	2784	1728
Convert Gray	4608	3648	4225	4032
Convert Reverse	4608	4224	6336	5760



최적화 아이디어 개요

가속 논리

- 잦은 메모리 접근(LDR, STR)은 data bus 점유로 다음 instruction을 느리게 만듦
 - Word, Half word, Byte단위 Load, Store는 모두 같은 사이클 점유
 - LDM, STM을 이용하면 Cycle 절약 가능.
 - 여러 Byte단위(이하 Block 단위)로 Load, Store하여 한 번에 많은 양을 block 단위로 처리
 - Block 단위에 맞지 않는 size의 경우는 일반적인 Byte단위로 처리
- 많은 Loop 횟수는 잦은 Branch로 인해 cycle 손해
 - 고정된 Loop는 횟수 반복문이 아닌 정적인 명령으로 설계(Loop Unrolling)
 - Block 단위 설계를 통한 Loop 횟수 절감 (Loop 횟수 = $(\text{Size} / \text{Block}) + (\text{Size} \% \text{Block})$)
- Binary 성질 이용
 - $255 - X = \sim X$
 - if $(X \geq 128)$, $X[7] = 1$
 - 위의 성질을 이용하여 4 Byte를 하나의 Register에서 연산.

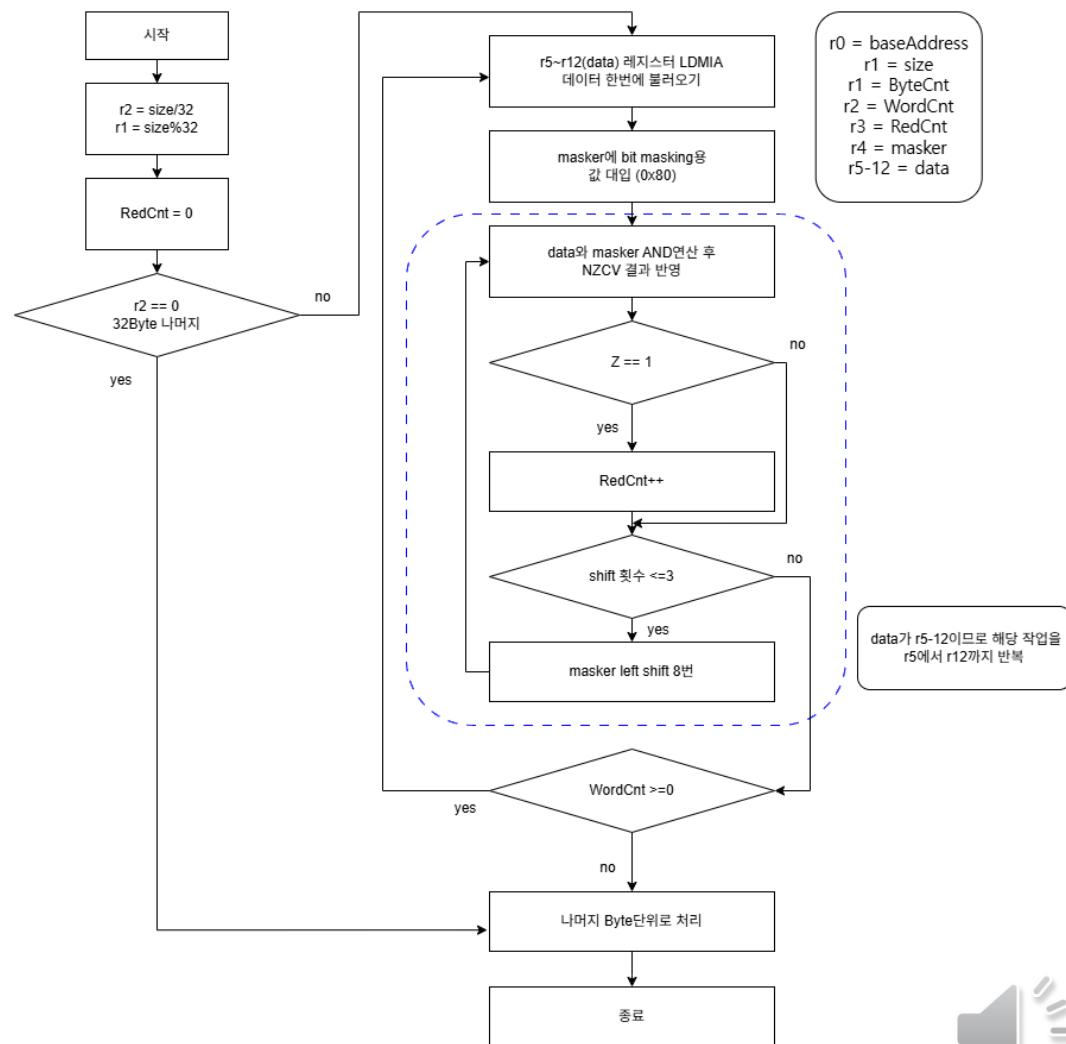


4-1. Red Pixel Count (optimization)

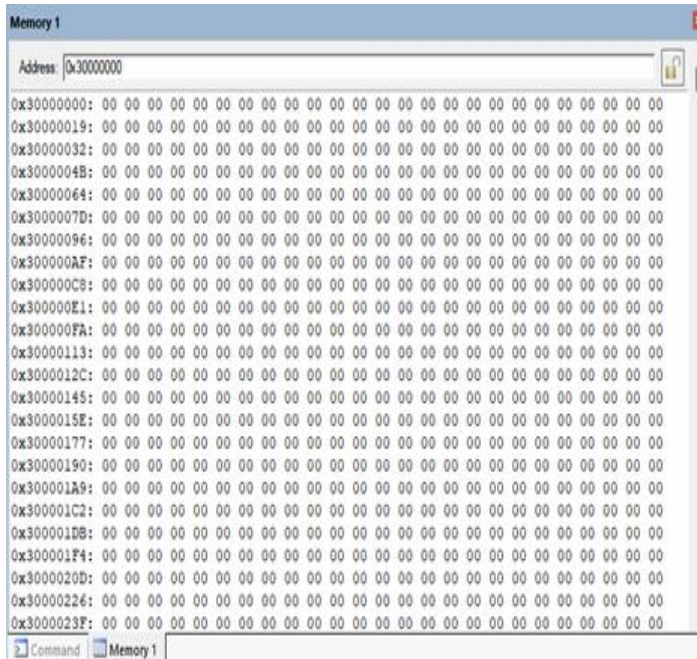
3	2	1	0
1000_0000	1000_0110	000_1100	1100_0100
0001_0000	0100_0110	000_1100	1110_0100

&

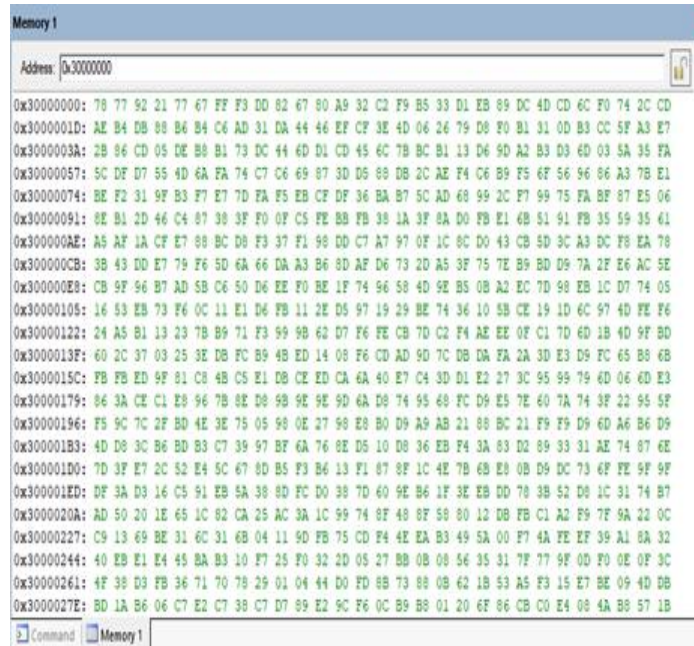
0000_0000	0000_0000	000_0000	1000_0000
-----------	-----------	----------	-----------



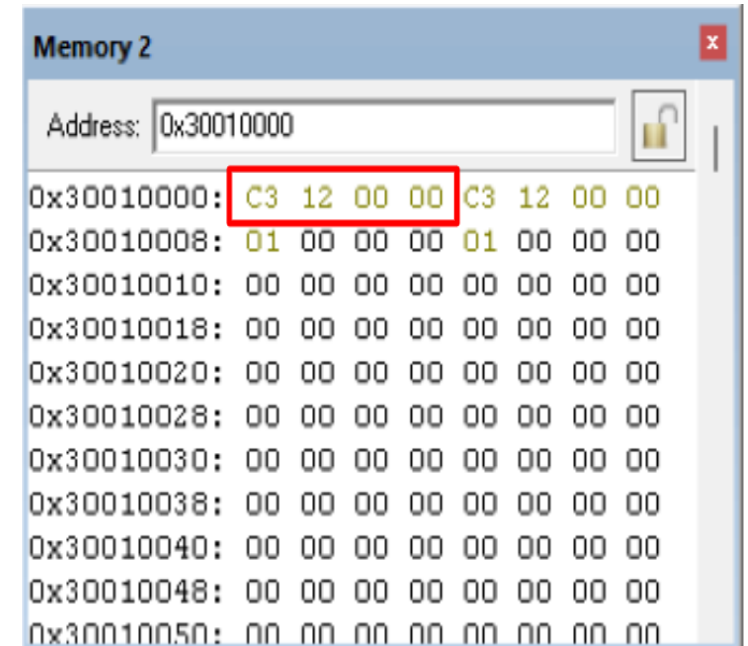
4-1. Red Pixel Count (optimization)



[실행 전 메모리맵]

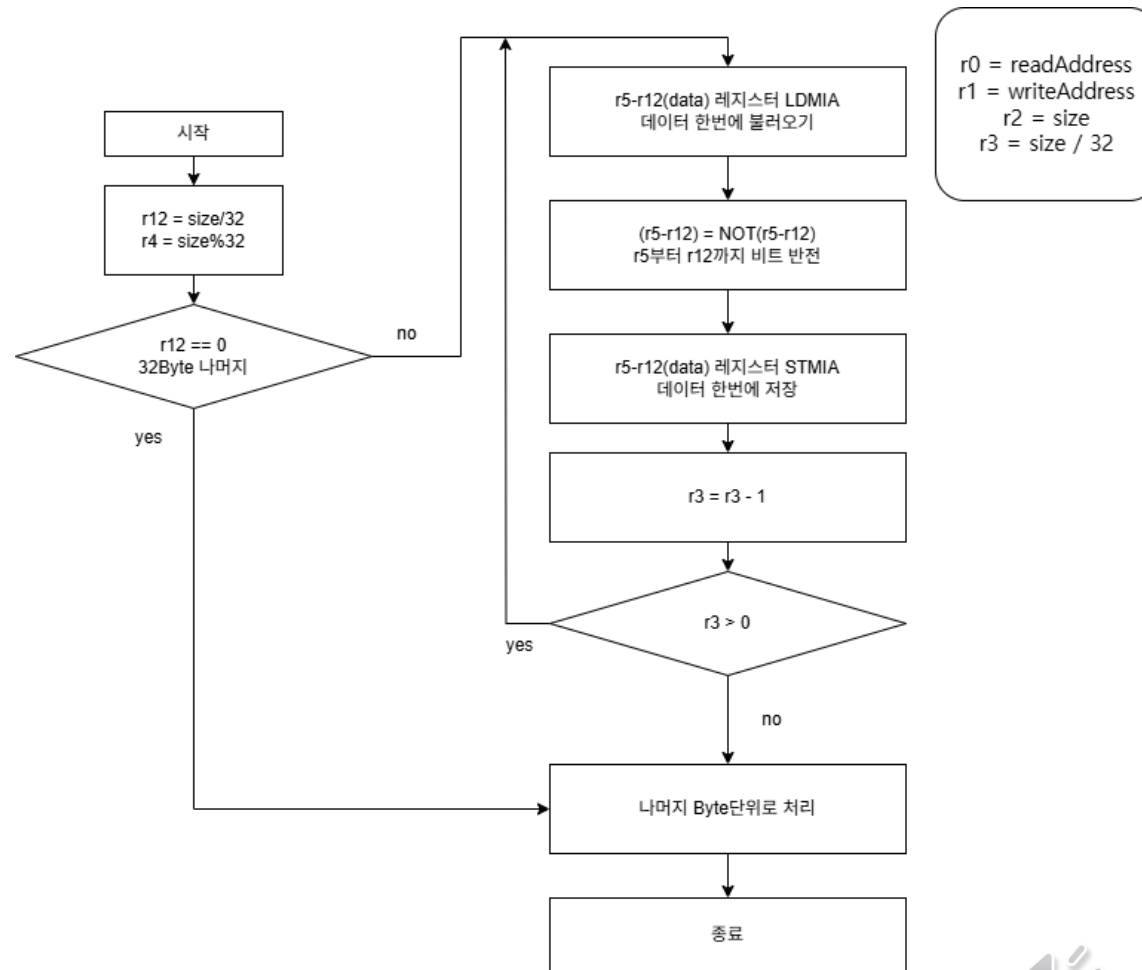
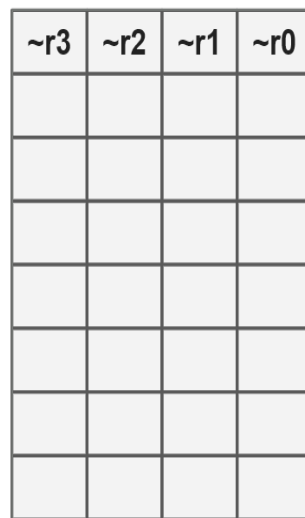


[실행 후 메모리맵]

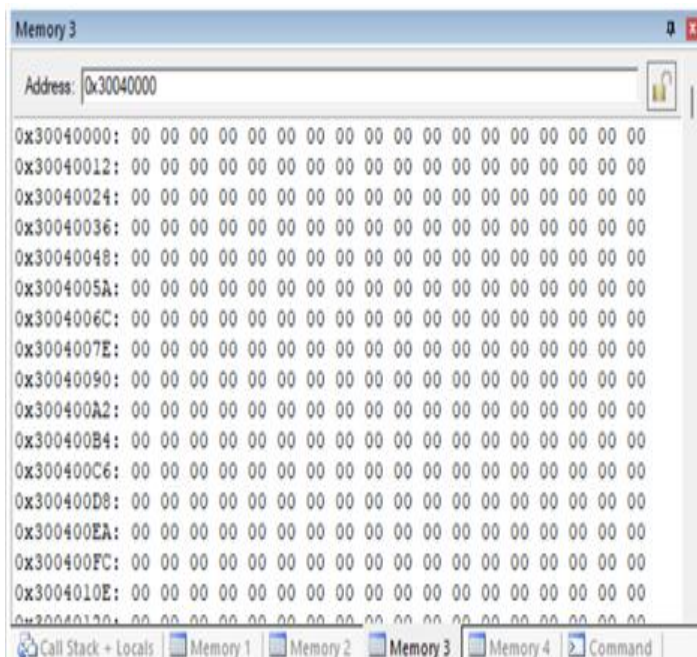


[0X30010000에 저장된 값]

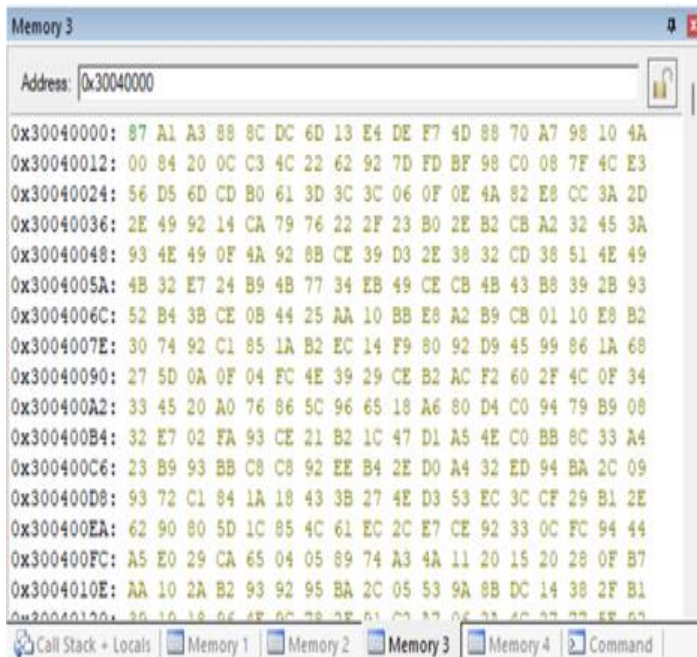
4-2. RGB convert (optimization)



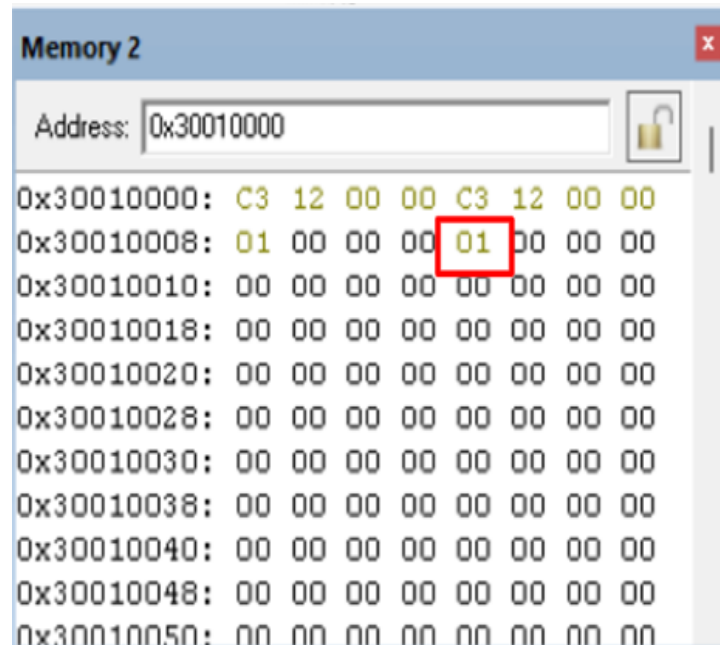
4-2. RGB convert (optimization)



[실행 전 메모리맵]



[실행 후 메모리맵]



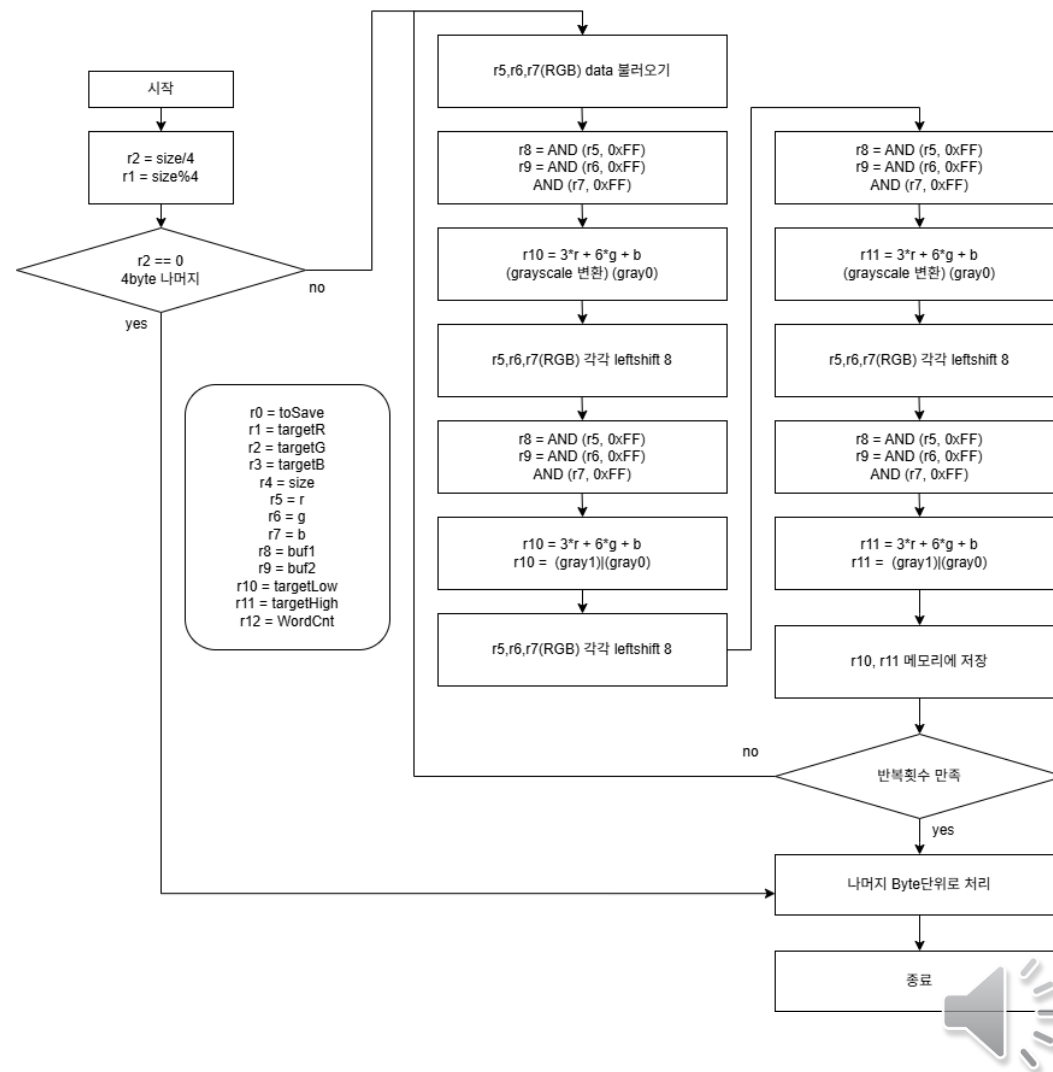
[bool isSame 실행결과]

4-3. Grayscale (optimization)

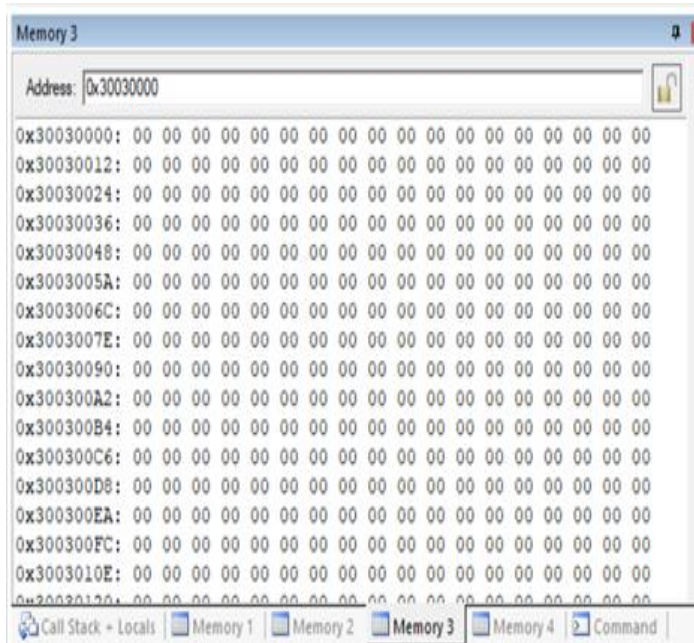
3	2	1	0
3	2	1	0
3	2	1	0



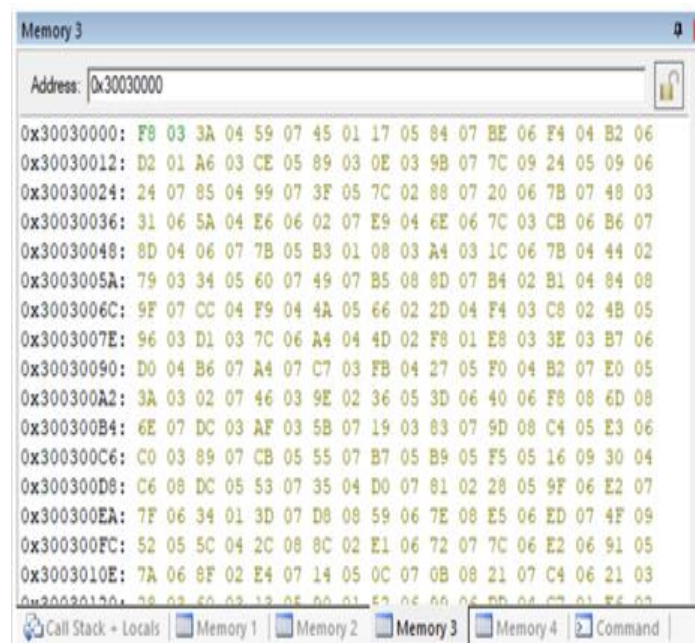
1	0	Low
3	2	High



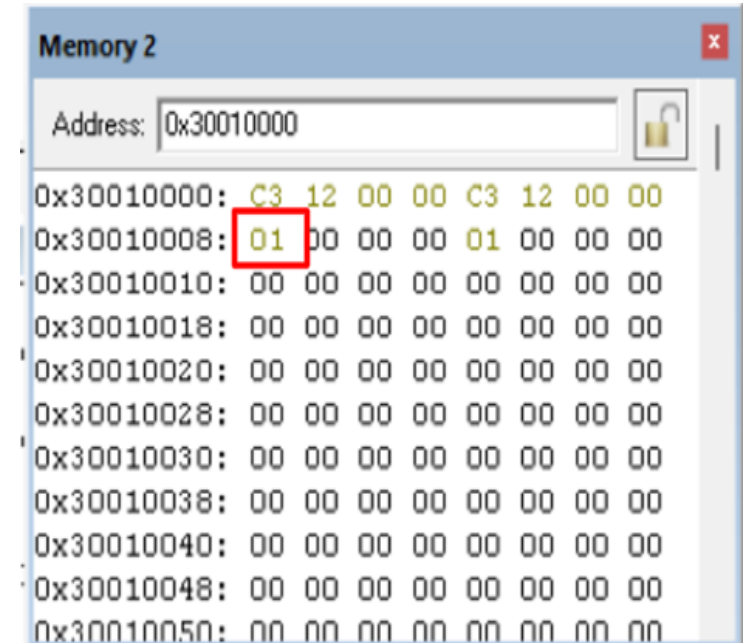
4-3. Grayscale (optimization)



[실행 전 메모리맵]



[실행 후 메모리맵]



[bool isSame 실행결과]

Performance Analyzer

Reset Show: Modules

Module/Function	Calls	Time(S...	Time(%)
ma_project_dj		40,477 ms	100%
source files/find_IDAT.c		53,020 us	0%
Startup.s		1,660 us	0%
../././Desktop/files/serial.c		0,380 us	0%
first grayrel.s		4,032 ms	10%
__asm_0x8C4	1	4,032 ms	10%
first reverserel.s		5,760 ms	14%
__asm_0x8A8	1	5,760 ms	14%
firstcountrel.s		1,728 ms	4%
__asm_0x888	1	1,728 ms	4%
source files/isSame.c		11,521 ms	28%
source files/main_reallo.c		2,740 us	0%
source files/convertReverse.s		0us	0%
source files/convertGray.s		0us	0%
source files/countRed.s		0us	0%
source files/reAllocation.c		17,378 ms	43%
countRedRel	1	2,784 ms	7%
convertReverseRel	1	6,336 ms	16%
convertGrayRel	1	4,225 ms	10%
reAllocation	1	4,032 ms	10%
source files/convertReverse...		0us	0%
source files/countRedRel.s		0us	0%
source files/convertGrayRel.s		0us	0%

[최적화 이전 Performance Analyzer 캡처]

Performance Analyzer

Reset Show: Modules

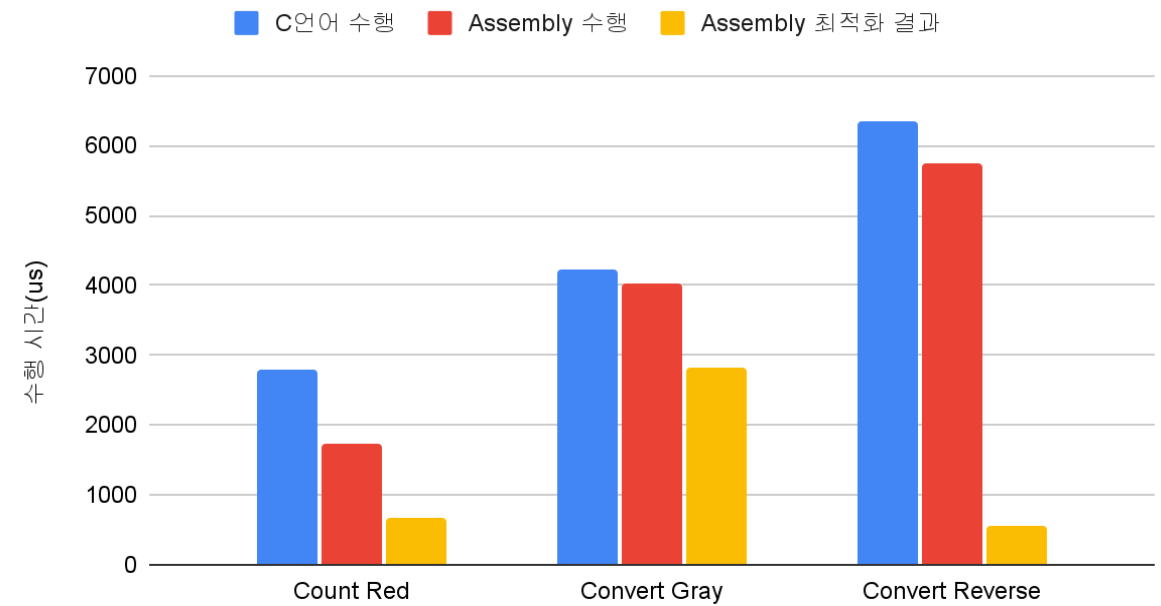
Module/Function	Calls	Time(S...	Time(%)
ma_project_dj		33,008 ms	100%
source files/find_IDAT.c		53,020 us	0%
Startup.s		1,660 us	0%
../././Desktop/files/serial.c		0,380 us	0%
source files/isSame.c		11,521 ms	35%
source files/main_reallo.c		2,740 us	0%
source files/convertReverse.s		0us	0%
source files/convertGray.s		0us	0%
source files/countRed.s		0us	0%
source files/reAllocation.c		17,378 ms	53%
countRedRel	1	2,784 ms	8%
convertReverseRel	1	6,336 ms	19%
convertGrayRel	1	4,225 ms	13%
reAllocation	1	4,032 ms	12%
source files/convertReverse...		558,620 us	2%
__asm_0x544	1	558,620 us	2%
source files/countRedRel.s		660,660 us	2%
__asm_0x5A8	1	660,660 us	2%
source files/convertGrayRel.s		2,833 ms	9%
__asm_0x770	1	2,833 ms	9%

[최적화 이후 Performance Analyzer 캡처]



(us)	C언어 수행	Assembly 수행	Assembly 최적화 결과	C대비 수행 시간
Count Red	2784	1728	660.66	23.73%
Convert Gray	4225	4032	2833	67.05%
Convert Reverse	6336	5760	558.62	8.82%

C언어 수행 결과, Assembly, 최적화 결과



#1 이미지 Save/Load 과정에서의 문제

-해결과정

Keil의 Debug 모드에서 이미지 로드 주소와 실행 시작 주소를 명확하게 지정하여 이미지 데이터가 메모리상 정확한 위치에 로드되도록 수정함.

#2 함수 호출 시 파라미터 전달 오류

-해결과정

C 기반 project 생성시, data를 load 한 영역과 임의로 지정한 포인터, 변수 영역이 서로를 침범하여 data가 오염되는 문제가 발생했었음. scatter 파일상에서 이 영역을 명확히 분리하여 Heap/Stack 간섭 문제를 해결. 변수 오염을 방지함.

#3 변수 오염 및 디버깅 어려움

-해결과정

relocation 이후 어셈블리 함수에서 5개 이상의 파라미터가 필요한데, APCS에 따라 r0-r3 4개 만의 레지스터만으로는 전달이 불가능했음. 따라서 의도된 동작을 하지 못했는데, APCS 표준에 대해서 이해하여 Stack을 통해 인자를 전달받도록 수정.



감사합니다

